

WEEK-4

Session-1

Problem Statement

Given a string *s* of even length, split it into two equal halves.

Return true if both halves contain the same number of vowels. Otherwise, return false.

Vowels are:

a, e, i, o, u, A, E, I, O, U

Code:

```
class Solution {  
    public boolean halvesAreAlike(String s) {  
        int mid = s.length() / 2;  
        int countA = 0;  
        int countB = 0;  
  
        for (int i = 0; i < mid; i++) {  
            if (isVowel(s.charAt(i))) {  
                countA++;  
            }  
        }  
  
        for (int i = mid; i < s.length(); i++) {  
            if (isVowel(s.charAt(i))) {  
                countB++;  
            }  
        }  
    }  
}
```

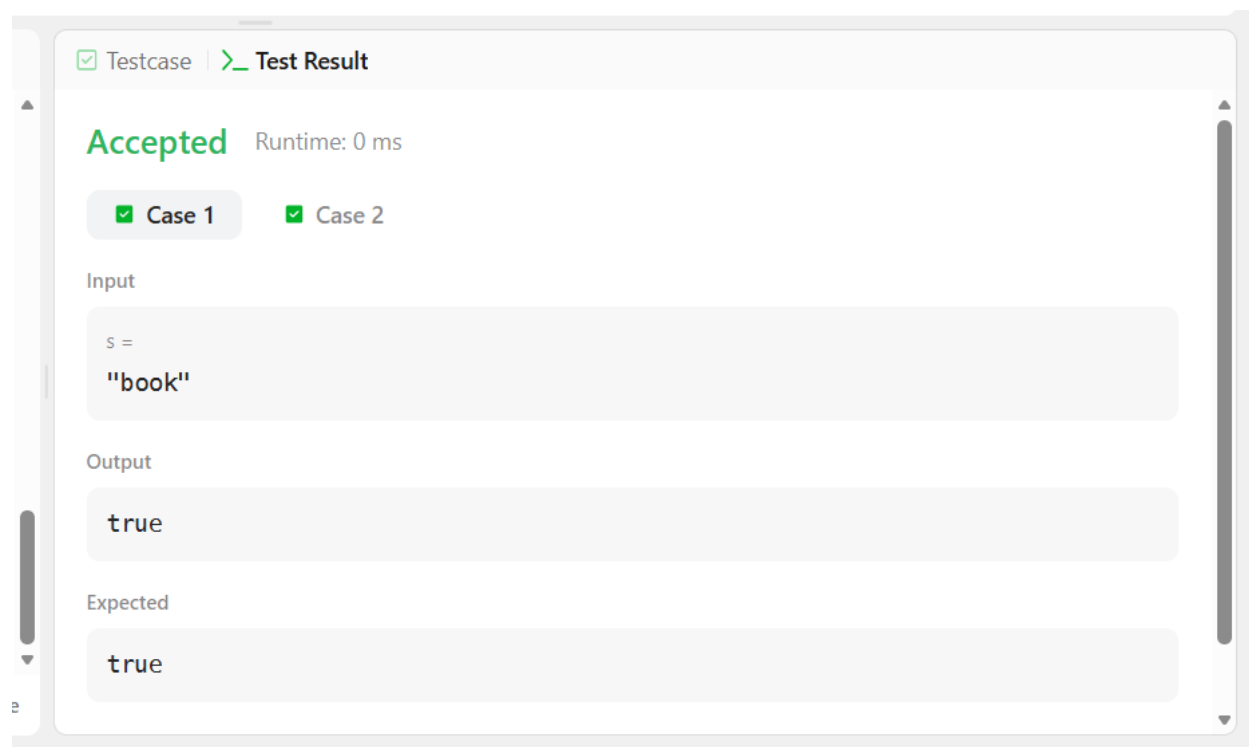
```

        return countA == countB;
    }

    public boolean isVowel(char ch) {
        return ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u'
            || ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U';
    }
}

```

Input/Output:



The screenshot shows a test result window with a tab labeled 'Test Result'. It displays 'Accepted' in green text with a runtime of '0 ms'. Below this, there are two test cases, 'Case 1' and 'Case 2', both marked as passed with green checkmarks. The 'Input' section shows 's = "book"'. The 'Output' section shows 'true'. The 'Expected' section also shows 'true'. The interface includes a scrollbar on the right and a small 'e' icon at the bottom left.

Problem Statement

Given a string, determine whether it is a lapindrome.

A lapindrome is a string whose left half and right half contain the same characters with the same frequencies. If the string has an odd length, ignore the middle character.

Print YES if the string is a lapindrome. Otherwise, print NO.

Code:

```
import java.util.*;
```

```
public class Main {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int t = sc.nextInt();
```

```
        while (t-- > 0) {
```

```
            String s = sc.next();
```

```
            int[] freq = new int[26];
```

```
            int n = s.length();
```

```
            for (int i = 0; i < n / 2; i++) {
```

```
                freq[s.charAt(i) - 'a']++;
```

```
            }
```

```
            for (int i = (n + 1) / 2; i < n; i++) {
```

```
                freq[s.charAt(i) - 'a']--;
```

```
            }
```

```
            boolean isLapindrome = true;
```

```
            for (int count : freq) {
```

```
        if (count != 0) {  
            isLapindrome = false;  
            break;  
        }  
    }  
}
```

```
    System.out.println(isLapindrome ? "YES" : "NO");  
}
```

```
    sc.close();  
}  
}
```

Input/Output:

Status : Successfully executed

Time:
0.0800 secs

Memory:
42.248 Mb

Sample Input

```
6
gaga
abcde
rotor
xyzxy
abbaab
ababc
```

Your Output

```
YES
NO
YES
YES
NO
NO
```

Session-2

Problem Statement

Given two triplets a and b , compare their corresponding elements.

- If $a[i] > b[i]$, Alice receives 1 point.
- If $a[i] < b[i]$, Bob receives 1 point.
- If $a[i] == b[i]$, neither receives a point.

Return Alice's score followed by Bob's score.

Code:

```
import java.io.*;
```

```
import java.util.*;

import java.util.stream.*;

import static java.util.stream.Collectors.toList;

import static java.util.stream.Collectors.joining;


class Result {

    public static List<Integer> compareTriplets(List<Integer> a, List<Integer> b) {

        int alice = 0;

        int bob = 0;


        for (int i = 0; i < 3; i++) {

            if (a.get(i) > b.get(i)) {

                alice++;

            } else if (a.get(i) < b.get(i)) {

                bob++;

            }

        }

        List<Integer> result = new ArrayList<>();

        result.add(alice);

        result.add(bob);


        return result;

    }

}
```

```

public class Solution {

    public static void main(String[] args) throws IOException {

        BufferedReader bufferedReader = new BufferedReader(
            new InputStreamReader(System.in)
        );

        BufferedWriter bufferedWriter = new BufferedWriter(
            new FileWriter(System.getenv("OUTPUT_PATH"))
        );

        List<Integer> a = Stream.of(
            bufferedReader.readLine().replaceAll("\\s+$", "").split(" ")
        )
        .map(Integer::parseInt)
        .collect(toList());

        List<Integer> b = Stream.of(
            bufferedReader.readLine().replaceAll("\\s+$", "").split(" ")
        )
        .map(Integer::parseInt)
        .collect(toList());

        List<Integer> result = Result.compareTriplets(a, b);

        bufferedWriter.write(
            result.stream()

```

```

        .map(Object::toString)
        .collect(joining(" "))
        + "\n"
    );

    bufferedReader.close();
    bufferedWriter.close();
}
}

```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0	Input (stdin) 1 5 6 7 2 3 6 10	Download
Sample Test case 1	Your Output (stdout) 1 1 1	
	Expected Output 1 1 1	Download

Problem Statement

Given an integer array `nums`, return `true` if any value appears at least twice.

Return `false` if every element in the array is distinct.

Code:

```
import java.util.*;

class Solution {
    public boolean containsDuplicate(int[] nums) {
        HashSet<Integer> set = new HashSet<>();

        for (int num : nums) {
            if (set.contains(num)) {
                return true;
            }
            set.add(num);
        }

        return false;
    }
}
```

Input/Output:

WEEK-5

Session-1

Problem Statement

Given an integer array `nums`, find the contiguous subarray with the largest sum and return its sum.

A subarray must contain at least one element.

Code:

```
class Solution {  
    public int maxSubArray(int[] nums) {  
        int currentSum = nums[0];  
        int maxSum = nums[0];  
  
        for (int i = 1; i < nums.length; i++) {  
            currentSum = Math.max(nums[i], currentSum + nums[i]);  
            maxSum = Math.max(maxSum, currentSum);  
        }  
  
        return maxSum;  
    }  
}
```

Input/Output:

✓ Testcase | > Test Result

Accepted Runtime: 0 ms

✓ Case 1 ✓ Case 2 ✓ Case 3

Input

```
nums =  
[-2,1,-3,4,-1,2,1,-5,4]
```

Output

```
6
```

Expected

```
6
```

PS: Maximum Subarray

Problem Statement

Given an integer array `nums`, find the contiguous subarray with the largest sum and return its sum.

Code:

```
import java.util.*;
```

```
public class Main {
```

```
    public static int maxSubArray(int[] nums) {
```

```
        int currentSum = nums[0];
```

```
        int maxSum = nums[0];
```

```
        for (int i = 1; i < nums.length; i++) {
```

```
currentSum = Math.max(nums[i], currentSum + nums[i]);
```

```
maxSum = Math.max(maxSum, currentSum);
```

```
}
```

```
return maxSum;
```

```
}
```

```
public static void main(String[] args) {
```

```
int[] nums = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
```

```
int result = maxSubArray(nums);
```

```
System.out.println(result);
```

```
}
```

```
}
```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

```
1 5
2 1 2 1 3 2
3 3 2
```

[Download](#)

Your Output (stdout)

```
1 2
```

Expected Output

```
1 2
```

[Download](#)

Session-2

PS: The Maximum Subarray

HackerRank | Dynamic Programming

Problem Statement

Given an array of integers, find the maximum possible sum among:

1. All non-empty subarrays — A contiguous portion of the array.
2. All non-empty subsequences — A selection of elements from the array, not necessarily contiguous.

Print the two maximum sums as space-separated integers.

Code:

```
import java.io.*;
```

```
import java.util.*;
```

```
public class Solution {
```

```
    public static int[] maxSubarray(int[] arr) {
```

```
int currentSum = arr[0];  
int maxSubarraySum = arr[0];  
  
int maxSubsequenceSum = arr[0];  
  
for (int i = 1; i < arr.length; i++) {  
  
    // Maximum Subarray Sum  
    currentSum = Math.max(  
        arr[i],  
        currentSum + arr[i]  
    );  
  
    maxSubarraySum = Math.max(  
        maxSubarraySum,  
        currentSum  
    );  
  
    // Maximum Subsequence Sum  
    if (arr[i] > 0) {  
        maxSubsequenceSum += arr[i];  
    }  
  
    // If current element is greater than  
    // the previous maximum negative value
```

```
        if (arr[i] > maxSubsequenceSum) {  
            maxSubsequenceSum = arr[i];  
        }  
    }  
}
```

```
return new int[] {  
    maxSubarraySum,  
    maxSubsequenceSum  
};  
}
```

```
public static void main(String[] args) throws IOException {
```

```
    BufferedReader br = new BufferedReader(  
        new InputStreamReader(System.in)  
    );
```

```
    int t = Integer.parseInt(br.readLine().trim());
```

```
    StringBuilder sb = new StringBuilder();
```

```
    while (t-- > 0) {
```

```
        int n = Integer.parseInt(br.readLine().trim());
```

```
        int[] arr = new int[n];
```

```
StringTokenizer st = new StringTokenizer(  
    br.readLine()  
);  
  
for (int i = 0; i < n; i++) {  
    arr[i] = Integer.parseInt(st.nextToken());  
}  
  
int[] result = maxSubarray(arr);  
  
sb.append(result[0])  
  .append(" ")  
  .append(result[1])  
  .append("\n");  
}  
  
System.out.print(sb.toString());  
}  
}
```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

```
2 4
3 1 2 3 4
4 6
5 2 -1 2 3 4 -5
```

Your Output (stdout)

```
1 10 10
2 10 11
```

Expected Output

```
1 10 10
2 10 11
```

[Download](#)

LeetCode 918 | Medium

Problem Statement

Given a circular integer array `nums`, return the maximum possible sum of a non-empty subarray.

A circular array means the end of the array connects to the beginning.

A subarray must contain at least one element, and each element can be used at most once.

Code:

```
class Solution {
    public int maxSubarraySumCircular(int[] nums) {
        int total = 0;
        int maxSum = nums[0];
        int minSum = nums[0];
        int currentMax = nums[0];
        int currentMin = nums[0];
```

```
for (int i = 0; i < nums.length; i++) {  
    if (i > 0) {  
        currentMax = Math.max(nums[i], currentMax + nums[i]);  
        maxSum = Math.max(maxSum, currentMax);  
  
        currentMin = Math.min(nums[i], currentMin + nums[i]);  
        minSum = Math.min(minSum, currentMin);  
    }  
  
    total += nums[i];  
}  
  
if (maxSum < 0) {  
    return maxSum;  
}  
  
return Math.max(maxSum, total - minSum);  
}  
}
```

Input/Output:

☒ Testcase | ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[1,-2,3,-2]

Output

3

Expected

3

Session-3

Problem Statement

Implement the `myAtoi(String s)` function, which converts a string into a 32-bit signed integer.

The algorithm follows these steps:

1. **Whitespace:** Ignore leading spaces.
2. **Sign:** Check for '-' or '+'. If neither is present, assume positive.
3. **Conversion:** Read digits until a non-digit character is encountered.
4. **Overflow:** If the result exceeds the 32-bit signed integer range, return the appropriate limit.

Code:

```
class Solution {  
    public int myAtoi(String s) {  
        int i = 0;
```

```
int n = s.length();
```

```
while (i < n && s.charAt(i) == ' ') {
```

```
    i++;
```

```
}
```

```
int sign = 1;
```

```
if (i < n && (s.charAt(i) == '+' || s.charAt(i) == '-')) {
```

```
    if (s.charAt(i) == '-') {
```

```
        sign = -1;
```

```
    }
```

```
    i++;
```

```
}
```

```
long result = 0;
```

```
while (i < n && Character.isDigit(s.charAt(i))) {
```

```
    int digit = s.charAt(i) - '0';
```

```
    result = result * 10 + digit;
```

```
    if (sign == 1 && result > Integer.MAX_VALUE) {
```

```
        return Integer.MAX_VALUE;
```

```
    }
```

```
    if (sign == -1 && -result < Integer.MIN_VALUE) {
```

```

        return Integer.MIN_VALUE;
    }

    i++;
}

return (int) (result * sign);
}
}

```

Input/Output:

☒ Testcase
 ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1
 ☒ Case 2
 ☒ Case 3
 ☒ Case 4
 ☒ Case 5

Input

s =
 "42"

Output

42

Expected

42

Contribute a testcase

Problem Statement

You are given a string containing characters A and B only.

Your task is to delete the minimum number of characters so that no two adjacent characters are the same.

Return the minimum number of deletions required.

Code:

```
import java.io.*;
```

```
import java.util.*;
```

```
public class Solution {
```

```
    public static int alternatingCharacters(String s) {
```

```
        int deletions = 0;
```

```
        for (int i = 1; i < s.length(); i++) {
```

```
            if (s.charAt(i) == s.charAt(i - 1)) {
```

```
                deletions++;
```

```
            }
```

```
        }
```

```
        return deletions;
```

```
    }
```

```
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader br = new BufferedReader(
```

```
            new InputStreamReader(System.in)
```

```
        );
```

```
int q = Integer.parseInt(br.readLine().trim());
```

```
StringBuilder sb = new StringBuilder();
```

```
while (q-- > 0) {
```

```
    String s = br.readLine().trim();
```

```
    int result = alternatingCharacters(s);
```

```
    sb.append(result).append("\n");
```

```
}
```

```
System.out.print(sb.toString());
```

```
}
```

```
}
```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

[Download](#)

```
1 5
2 AAAA
3 BBBBB
4 ABABABAB
5 BABABA
6 AAABBB
```

Your Output (stdout)

```
1 3
2 4
3 0
```

Session-4

Problem Statement

Given a string *s*, find the length of the longest substring without duplicate characters.

A substring is a contiguous sequence of characters in a string.

Code:

```
import java.util.HashSet;
```

```
class Solution {
```

```
    public int lengthOfLongestSubstring(String s) {
```

```
        HashSet<Character> set = new HashSet<>();
```

```
        int left = 0;
```

```
        int maxLength = 0;
```

```
for (int right = 0; right < s.length(); right++) {  
  
    char ch = s.charAt(right);  
  
    while (set.contains(ch)) {  
        set.remove(s.charAt(left));  
        left++;  
    }  
  
    set.add(ch);  
  
    int currentLength = right - left + 1;  
    maxLength = Math.max(maxLength, currentLength);  
}  
  
return maxLength;  
}  
}
```

Input/Output:

Testcase

Test Result

Accepted

Runtime: 0 ms

Case 1

Case 2

Case 3

Input

S =
"abcabcbb"

Output

3

Expected

3

Contribute a testcase

Find and Replace Pattern

Approach: Two-Way Character Mapping

A word matches the pattern when:

- Each character in the pattern maps to only one character in the word.
- Two different pattern characters cannot map to the same word character.

We use two arrays:

- `patternToWord[26]`
- `wordToPattern[26]`

A value of -1 means that the character has not been mapped yet.

Code:

☒ Testcase | [Test Result](#)

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

words =
["abc", "deq", "mee", "aqq", "dkd", "ccc"]

pattern =
"abb"

Output

["mee", "aqq"]

Session-5

String Matching in an Array

Approach

For every word, check whether it appears as a substring in any other word.

- Use `contains()` to check substring.
- Skip comparing a word with itself.
- If found in another word, add it to the result.

Code:

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
class Solution {
```

```
    public List<String> stringMatching(String[] words) {
```

```
List<String> result = new ArrayList<>();
```

```
for (int i = 0; i < words.length; i++) {
```

```
    for (int j = 0; j < words.length; j++) {
```

```
        if (i == j) {
```

```
            continue;
```

```
        }
```

```
        if (words[j].contains(words[i])) {
```

```
            result.add(words[i]);
```

```
            break;
```

```
        }
```

```
    }
```

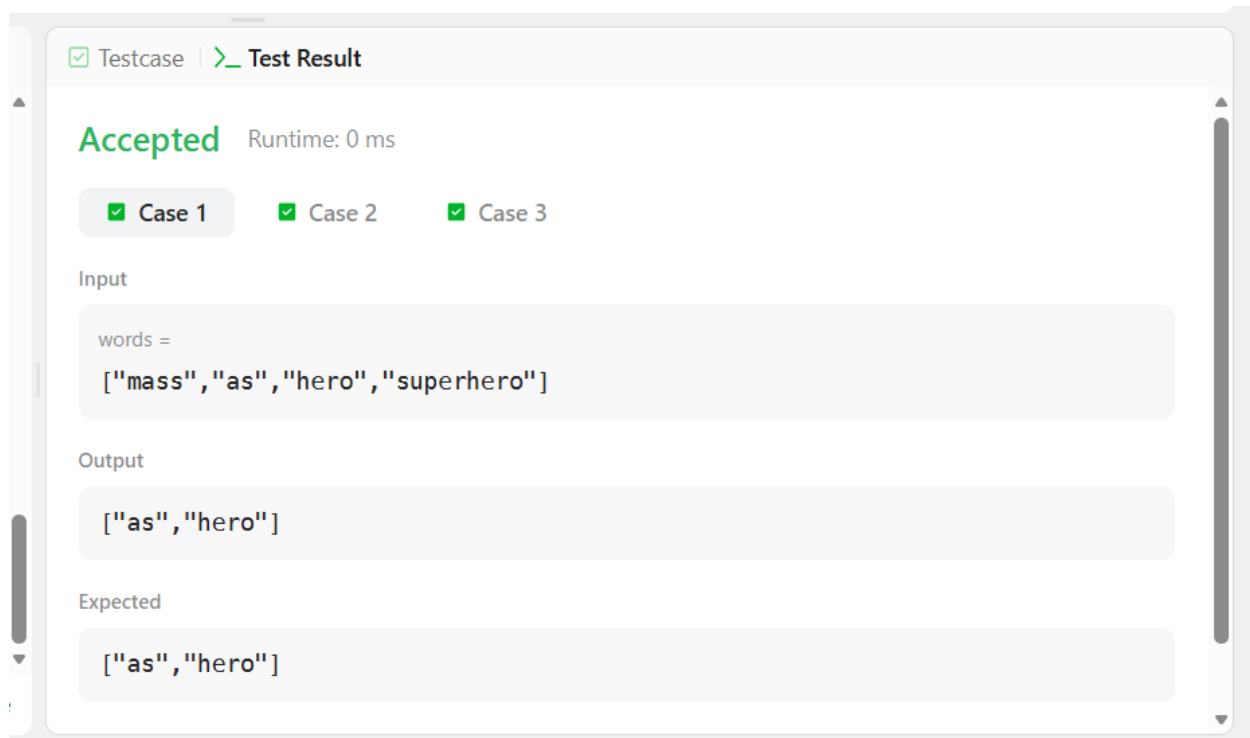
```
}
```

```
return result;
```

```
}
```

```
}
```

Input/Output:



PS: Naive Algorithm for Pattern Searching

Problem Statement

Given a text string of length n and a pattern string of length m , find all starting indices where the pattern occurs in the text.

The pattern may occur multiple times, including overlapping occurrences.

Code:

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
class Solution {
```

```
    public List<Integer> search(String text, String pattern) {
```

```
        List<Integer> result = new ArrayList<>();
```

```
int n = text.length();  
int m = pattern.length();  
  
for (int i = 0; i <= n - m; i++) {  
  
    int j;  
  
    for (j = 0; j < m; j++) {  
  
        if (text.charAt(i + j) != pattern.charAt(j)) {  
            break;  
        }  
    }  
}  
  
// If j reached m, the complete pattern matched  
if (j == m) {  
    result.add(i);  
}  
}  
  
return result;  
}  
}
```

Input/Output:

Output

```
0 9 12
```

WEEK-6

Session-1

PS: HackerRank — String Similarity

Approach: Z Algorithm

For each suffix of the string, we need to find the length of the longest prefix that matches the original string.

The Z Algorithm calculates an array where:

$z[i]$ = length of the longest substring starting at i
that matches the prefix of the string

The answer is:

$\text{string.length()} + \text{sum of all } z[i]$

We include the complete string itself, whose similarity is its full length.

Code:

```
import java.io.*;
```

```
import java.util.*;
```

```
public class Solution {
```

```
    public static long stringSimilarity(String s) {
```

```
        int n = s.length();
```

```
        int[] z = new int[n];
```

```
int left = 0;
```

```
int right = 0;
```

```
long answer = n;
```

```
for (int i = 1; i < n; i++) {
```

```
    if (i <= right) {
```

```
        z[i] = Math.min(right - i + 1, z[i - left]);
```

```
    }
```

```
    while (i + z[i] < n &&
```

```
        s.charAt(z[i]) == s.charAt(i + z[i])) {
```

```
        z[i]++;
```

```
    }
```

```
    if (i + z[i] - 1 > right) {
```

```
        left = i;
```

```
        right = i + z[i] - 1;
```

```
    }
```

```
    answer += z[i];
```

```
}
```

```
return answer;
```

```
}
```

```
public static void main(String[] args) throws IOException {
```

```
    BufferedReader br =
```

```
        new BufferedReader(new InputStreamReader(System.in));
```

```
    int t = Integer.parseInt(br.readLine().trim());
```

```
    StringBuilder output = new StringBuilder();
```

```
    while (t-- > 0) {
```

```
        String s = br.readLine().trim();
```

```
        long result = stringSimilarity(s);
```

```
        output.append(result).append('\n');
```

```
    }
```

```
    System.out.print(output);
```

```
}
```

```
}
```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

Input (stdin)

[Download](#)

```
1 2
2 ababaa
3 aa
```

Your Output (stdout)

```
1 11
2 3
```

Expected Output

[Download](#)

```
1 11
2 3
```

Repeated Substring Pattern

Approach

A string can be formed by repeating a substring if:

1. Its length is divisible by the substring length.
2. The substring repeats to form the complete string.

We try every possible substring length from 1 to $n / 2$.

Code:

```
class Solution {

    public boolean repeatedSubstringPattern(String s) {

        int n = s.length();

        for (int len = 1; len <= n / 2; len++) {

            // The string length must be divisible
```

```

// by the repeating substring length
if (n % len != 0) {
    continue;
}

boolean matches = true;

for (int i = len; i < n; i++) {

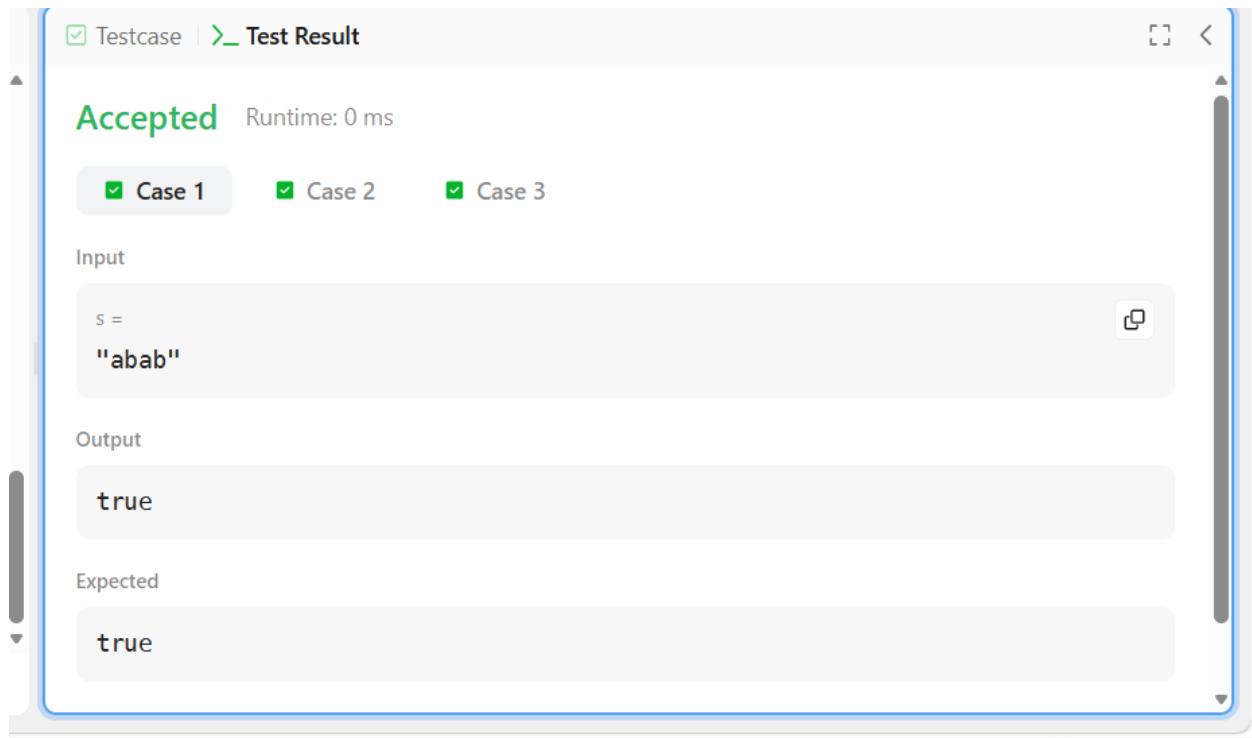
    if (s.charAt(i) != s.charAt(i % len)) {
        matches = false;
        break;
    }
}

if (matches) {
    return true;
}

return false;
}
}

```

Input/Output:



Session-2

Problem Statement

Given two strings, determine if they share a common substring. A substring may be as small as one character.

Complete the function `twoStrings`.

Return:

- YES if the strings share a common substring.
- NO otherwise.

Code:

```
import java.io.*;
```

```
import java.util.*;
```

```
public class Solution {
```

```
public static String twoStrings(String s1, String s2) {
```

```
    boolean[] present = new boolean[26];
```

```
    for (int i = 0; i < s1.length(); i++) {
```

```
        present[s1.charAt(i) - 'a'] = true;
```

```
    }
```

```
    for (int i = 0; i < s2.length(); i++) {
```

```
        if (present[s2.charAt(i) - 'a']) {
```

```
            return "YES";
```

```
        }
```

```
    }
```

```
    return "NO";
```

```
}
```

```
public static void main(String[] args) throws IOException {
```

```
    BufferedReader br =
```

```
        new BufferedReader(new InputStreamReader(System.in));
```

```
    int t = Integer.parseInt(br.readLine().trim());
```

```
    StringBuilder output = new StringBuilder();
```

```

while (t-- > 0) {

    String s1 = br.readLine().trim();

    String s2 = br.readLine().trim();

    output.append(twoStrings(s1, s2)).append('\n');

}

System.out.print(output);

}
}

```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

[Download](#)

```

1 2
2 hello
3 world
4 hi
5 world

```

Your Output (stdout)

```

1 YES
2 NO

```

Expected Output

[Download](#)

Problem Statement

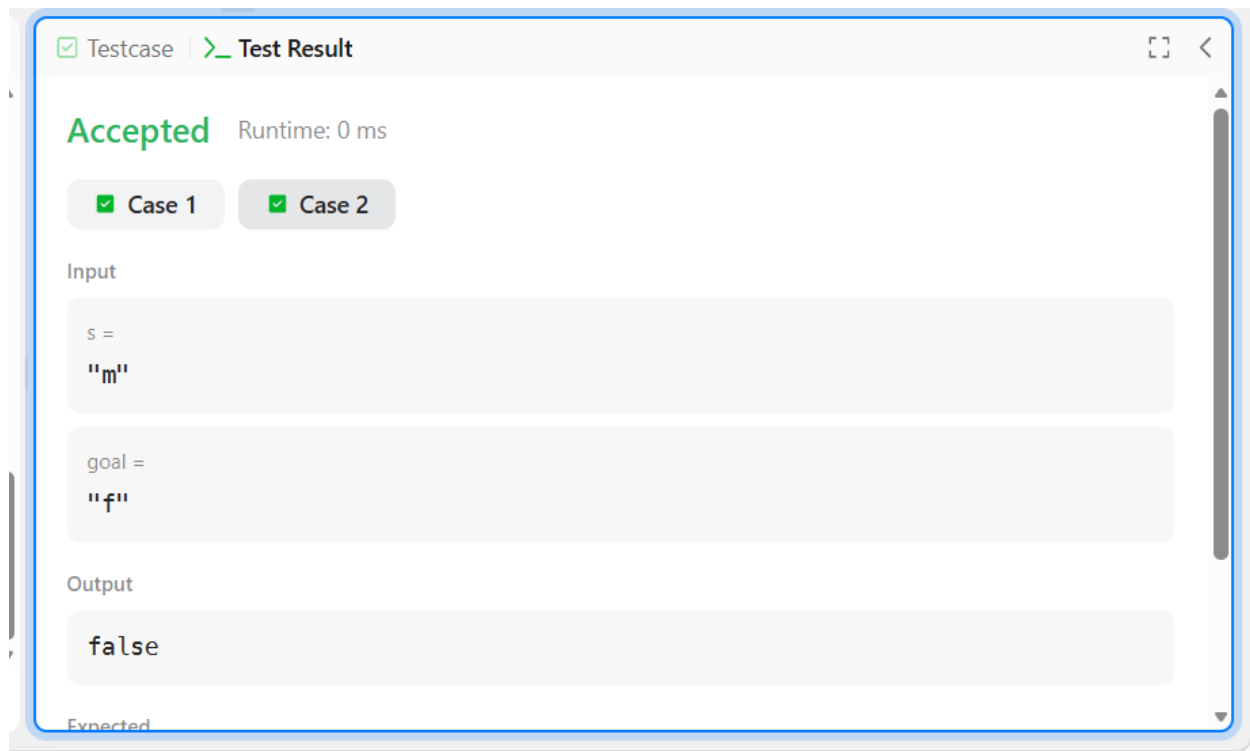
Given two strings s and $goal$, return true if and only if s can become $goal$ after some number of shifts on s .

A shift moves the leftmost character of s to the rightmost position.

Code:

```
class Solution {  
  
    public boolean rotateString(String s, String goal) {  
  
        if (s.length() != goal.length()) {  
            return false;  
        }  
  
        String doubled = s + s;  
  
        return doubled.contains(goal);  
    }  
}
```

Input/Output:



Session-3

Problem Statement

Given a string `s` containing a received signal, determine how many characters differ from the expected repeated message "SOS".

The expected message is formed by repeating "SOS" until it reaches the length of `s`.

Return the number of changed characters.

Code:

```
import java.io.*;
```

```
public class Solution {
```

```
    public static int marsExploration(String s) {
```

```
        String pattern = "SOS";
```

```
        int changes = 0;
```

```

for (int i = 0; i < s.length(); i++) {

    if (s.charAt(i) != pattern.charAt(i % 3)) {
        changes++;
    }
}

return changes;
}

public static void main(String[] args) throws IOException {

    BufferedReader br =
        new BufferedReader(new InputStreamReader(System.in));

    String s = br.readLine().trim();

    System.out.println(marsExploration(s));
}
}

```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

Input (stdin)

[Download](#)

✓ Sample Test case 1

1 S0SSPSSQSS0R

✓ Sample Test case 2

Your Output (stdout)

1 3

Expected Output

1 3

[Download](#)

Problem Statement

Given two strings s and p , return an array of all the starting indices of p 's anagrams in s .

You may return the answer in any order.

Code:

```
import java.util.*;
```

```
class Solution {
```

```
    public List<Integer> findAnagrams(String s, String p) {
```

```
        List<Integer> result = new ArrayList<>();
```

```
        if (s.length() < p.length()) {
```

```
            return result;
```

```
}
```

```
int[] count = new int[26];
```

```
for (int i = 0; i < p.length(); i++) {
```

```
    count[p.charAt(i) - 'a']++;
```

```
}
```

```
int windowSize = p.length();
```

```
for (int i = 0; i < s.length(); i++) {
```

```
    count[s.charAt(i) - 'a']--;
```

```
    if (i >= windowSize) {
```

```
        count[s.charAt(i - windowSize) - 'a']++;
```

```
    }
```

```
    if (i >= windowSize - 1 && isAnagram(count)) {
```

```
        result.add(i - windowSize + 1);
```

```
    }
```

```
}
```

```
return result;
```

```
}
```

```
private boolean isAnagram(int[] count) {  
  
    for (int value : count) {  
        if (value != 0) {  
            return false;  
        }  
    }  
  
    return true;  
}  
}
```

Input/Output:

The screenshot shows a test result window with a title bar containing 'Testcase' and 'Test Result'. The main content area displays 'Accepted' in green text, followed by 'Runtime: 0 ms'. Below this, there are two tabs: 'Case 1' and 'Case 2', both marked with a green checkmark. The 'Input' section shows two text boxes: the first contains 's =' followed by '"cbaebabacd"', and the second contains 'p =' followed by '"abc"'. The 'Output' section shows a text box containing '[0,6]'. At the bottom, there is a label 'Expected' followed by a blank text box. A vertical scrollbar is visible on the right side of the window.

Session-4

Problem Statement

Given a string of lowercase English letters, determine the index of a character that can be removed to make the string a palindrome.

If the string is already a palindrome or no single character removal can make it a palindrome, return -1.

If multiple answers are possible, return any valid index.

Code:

```
import java.util.*;
```

```
public class Solution {
```

```
    public static int palindromeIndex(String s) {
```

```
        int left = 0;
```

```
        int right = s.length() - 1;
```

```
        while (left < right) {
```

```
            if (s.charAt(left) != s.charAt(right)) {
```

```
                if (isPalindrome(s, left + 1, right)) {
```

```
                    return left;
```

```
                }
```

```
                if (isPalindrome(s, left, right - 1)) {
```

```
                    return right;
```

```
                }
```

```
        return -1;
```

```
}
```

```
left++;
```

```
right--;
```

```
}
```

```
return -1;
```

```
}
```

```
private static boolean isPalindrome(String s, int left, int right) {
```

```
    while (left < right) {
```

```
        if (s.charAt(left) != s.charAt(right)) {
```

```
            return false;
```

```
        }
```

```
        left++;
```

```
        right--;
```

```
    }
```

```
    return true;
```

```
}
```

```
public static void main(String[] args) {
```

```
    Scanner sc = new Scanner(System.in);
```

```
    int q = sc.nextInt();
```

```

while (q-- > 0) {

    String s = sc.next();

    System.out.println(palindromeIndex(s));

}

sc.close();

}
}

```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

 Sample Test case 0	Compiler Message Success
Input (stdin) 1 3 2 aaab 3 baa 4 aaa	Download
Your Output (stdout) 1 3 2 0	

Problem Statement

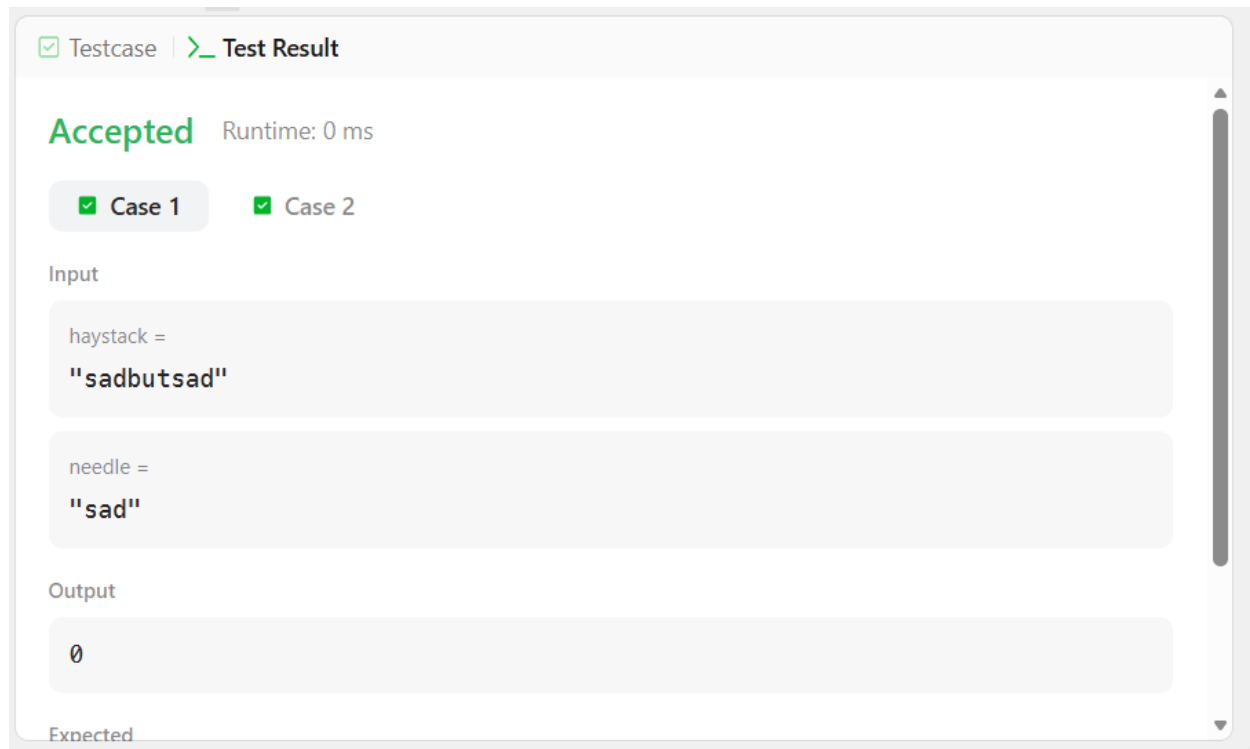
Given two strings haystack and needle, return the index of the first occurrence of needle in haystack.

If needle is not part of haystack, return -1.

Code:

```
class Solution {  
    public int strStr(String haystack, String needle) {  
        int n = haystack.length();  
        int m = needle.length();  
  
        for (int i = 0; i <= n - m; i++) {  
            int j = 0;  
  
            while (j < m && haystack.charAt(i + j) == needle.charAt(j)) {  
                j++;  
            }  
  
            if (j == m) {  
                return i;  
            }  
        }  
  
        return -1;  
    }  
}
```

Input/Output:



Session-5

Problem Statement

Given a string *s*, return the longest palindromic substring in *s*.

Code:

```
class Solution {  
    public String longestPalindrome(String s) {  
        if (s.length() < 2) {  
            return s;  
        }  
  
        int start = 0;  
        int end = 0;
```

```

for (int i = 0; i < s.length(); i++) {
    int len1 = expandAroundCenter(s, i, i);
    int len2 = expandAroundCenter(s, i, i + 1);

    int len = Math.max(len1, len2);

    if (len > end - start + 1) {
        start = i - (len - 1) / 2;
        end = i + len / 2;
    }
}

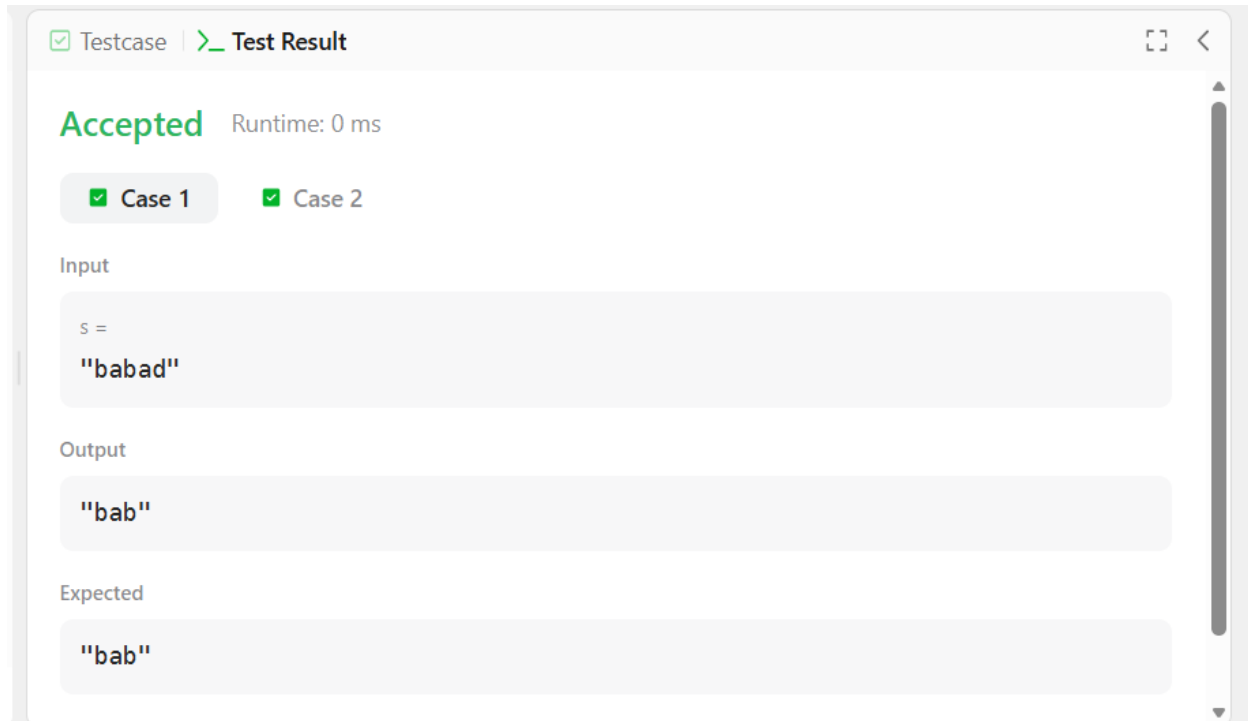
return s.substring(start, end + 1);
}

private int expandAroundCenter(String s, int left, int right) {
    while (left >= 0 &&
        right < s.length() &&
        s.charAt(left) == s.charAt(right)) {
        left--;
        right++;
    }

    return right - left - 1;
}
}

```

Input/Output:



The screenshot shows a web-based test runner interface. At the top, there are tabs for 'Testcase' and 'Test Result'. The 'Test Result' tab is active, displaying a green 'Accepted' status and a runtime of '0 ms'. Below this, two test cases are listed: 'Case 1' and 'Case 2', both marked as passed with green checkmarks. The 'Input' section shows 's =' followed by a text box containing '"babad"'. The 'Output' section shows a text box containing '"bab"'. The 'Expected' section shows a text box containing '"bab"'. A vertical scrollbar is visible on the right side of the interface.

Problem Statement

Given a string s of length n , consider all n circular rotations of s .

For each rotation, find the maximum length of a palindromic substring and print that length on a new line.

Code:

```
import java.util.*;
```

```
public class Solution {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int n = sc.nextInt();
```

```
String s = sc.next();
```

```
String doubled = s + s;
```

```
for (int start = 0; start < n; start++) {
```

```
    int maxLength = 1;
```

```
    for (int center = start; center < start + n; center++) {
```

```
        int len1 = expand(doubled, center, center, start, start + n - 1);
```

```
        int len2 = expand(doubled, center, center + 1, start, start + n - 1);
```

```
        maxLength = Math.max(maxLength, Math.max(len1, len2));
```

```
    }
```

```
    System.out.println(maxLength);
```

```
}
```

```
sc.close();
```

```
}
```

```
private static int expand(
```

```
    String s,
```

```
    int left,
```

```
    int right,
```

```
    int minIndex,
```

```
    int maxIndex
```

```

    ){
        while (left >= minIndex &&
            right <= maxIndex &&
            s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }

        return right - left - 1;
    }
}

```

Input/Output:

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

✓ Sample Test case 0

✓ Sample Test case 1

✓ Sample Test case 2

Input (stdin)

[Download](#)

```

1 13
2 aaaaabbbbbaaaa

```

Your Output (stdout)

```

1 12
2 12
3 10
4 8
5 8
6 9
7 11

```

